Quiero que construyas un PROTOTIPO FUNCIONAL completo de un instalador de sistema operativo llamado DylanOS.

IMPORTANTE:
Las imágenes adjuntas son la referencia visual principal. Analiza TODAS las imágenes antes de construir la interfaz.

NO quiero una copia genérica de Calamares.
Quiero reproducir la identidad visual de DylanOS mostrada en los diseños.

==========
1. IDENTIDAD VISUAL
==========

Usa exactamente la estética general de los diseños:

- Fondo principal negro / casi negro.
- Rojo DylanOS como color de acento.
- Blanco para información principal.
- Gris para información secundaria.
- Bordes redondeados.
- Paneles oscuros tipo glassmorphism.
- Transparencias sutiles.
- Sombras suaves.
- Bordes muy discretos.
- Iluminación/glow rojo muy moderado.
- Diseño premium, moderno y tecnológico.
- Mucho espacio negativo.
- Tipografía limpia y legible.
- Iconografía minimalista.
- No utilizar colores aleatorios.
- No introducir elementos visuales que contradigan el diseño.

El resultado debe parecer un producto propio llamado DylanOS,
NO Debian con un tema rojo.

==========
2. ESTRUCTURA GENERAL
==========

El instalador tendrá navegación por etapas mediante una barra lateral.

Etapas:

1. Bienvenido
2. Idioma
3. Teclado
4. Particionado

5. Usuario
6. Resumen
7. Instalar DylanOS
8. Finalizar

La etapa activa debe destacarse visualmente en rojo.

Las etapas completadas deben mostrar un estado visual de completado.

Las etapas futuras deben aparecer desactivadas o pendientes.

Los botones inferiores deben permitir:

- Atrás
- Siguiente
- Cancelar

Durante la instalación real, algunas acciones deben estar bloqueadas.

==========
3. PANTALLA 1 — BIENVENIDO
==========

Reproducir el diseño mostrado en la imagen de bienvenida.

Debe contener:

- Logo DylanOS.
- "Instalar DylanOS".
- Versión.
- Título "Bienvenido a DylanOS".
- Texto introductorio.
- Imagen principal de DylanOS.
- Tarjetas:
  - Rápido
  - Seguro
  - Personalizable
- Panel lateral de resumen del sistema.
- Selector de idioma.
- Navegación lateral.
- Botón Siguiente.

FUNCIONALIDAD DEL PROTOTIPO:

- Siguiente lleva a Idioma.
- Cancelar muestra un diálogo de confirmación.
- Los elementos visuales deben tener estados hover/pressed.
- Las tarjetas pueden tener animaciones sutiles.

DATOS DINÁMICOS:

El hardware mostrado en esta pantalla NO debe considerarse información fija.

Crear una estructura de datos simulada:

device
cpu
ram
storage
gpu

Usar datos de ejemplo durante el prototipo.

IMPORTANTE:
Diseñar el código de forma que posteriormente podamos reemplazar
los datos simulados por datos reales provenientes de Linux.

==========
4. PANTALLA 2 — IDIOMA
==========

Reproducir exactamente el lenguaje visual de los diseños.

Permitir seleccionar:

- Español (Latinoamérica)
- Español (España)
- English
- Otros idiomas como opción futura.

El selector debe ser funcional dentro del prototipo.

Cuando se seleccione un idioma:

- actualizar visualmente la selección;
- actualizar el idioma mostrado;
- guardar la selección en el estado del prototipo.

El idioma real del sistema será conectado posteriormente a Calamares.

==========
5. PANTALLA 3 — TECLADO
==========

Crear una pantalla funcional para seleccionar distribución de teclado.

Mostrar:

- idioma;
- distribución;
- teclado visual de referencia;
- lista de distribuciones;
- opción de probar teclado.

Permitir seleccionar una distribución.

Guardar la selección en el estado global.

El teclado real del sistema será conectado posteriormente a Linux/Calamares.

==========
6. PANTALLA 4 — PARTICIONADO
==========

Crear una interfaz visual de particionado inspirada directamente
en el diseño proporcionado.

Mostrar:

- disco;
- capacidad;
- particiones;
- espacio disponible;
- sistema de archivos;
- punto de montaje;
- esquema de particionado.

Crear dos modos de prototipo:

A) Instalación automática
B) Particionado manual

La interfaz debe reaccionar a las selecciones.

NO realizar operaciones reales sobre discos.

Todo debe ser SIMULADO.

IMPORTANTE:
La parte visual debe estar preparada para que posteriormente
Calamares proporcione los datos reales de los discos.

==========
7. PANTALLA 5 — USUARIO

==========

Esta pantalla fue añadida después del diseño inicial.

Debe solicitar:

- Nombre completo
- Nombre de usuario
- Contraseña
- Confirmación de contraseña
- Contraseña de root
- Nombre de la máquina

Mostrar validaciones visuales.

Ejemplos:

Nombre de usuario:
✓ válido

Contraseña:
✓ coincide

Confirmación:
× las contraseñas no coinciden

Nombre de máquina:
✓ válido

No almacenar contraseñas reales en ningún lugar del prototipo.

Utilizar únicamente datos temporales de demostración.

El prototipo debe guardar el resto de los valores para utilizarlos posteriormente en la pantalla de resumen.

==========
8. PANTALLA 6 — RESUMEN
==========

Mostrar un resumen completo de las decisiones realizadas.

Por ejemplo:

Idioma
Español (Latinoamérica)

Teclado

Español Latinoamericano

Disco
/dev/sda

Particionado
Instalación automática

Usuario
dylan

Nombre del equipo
DylanOS-PC

También mostrar un resumen del hardware.

IMPORTANTE:

Los datos de hardware son SIMULADOS en el prototipo.

Preparar la estructura para reemplazarlos posteriormente por:

- CPU real
- GPU real
- RAM real
- almacenamiento real
- modelo real del dispositivo

Agregar botones para volver a modificar cada configuración.

==========
9. PANTALLA 7 — INSTALANDO DYLANOS
==========

Esta pantalla NO debe ser una simple animación.

Crear una simulación funcional de instalación.

Mostrar:

- "Instalando DylanOS"
- progreso general;
- etapa actual;
- paquete/proceso actual;
- información del sistema;
- pasos de instalación.

Ejemplo:

Preparando
✓

Copiando archivos
✓

Instalando paquetes
●

Configurando sistema
○

Finalizando
○

El porcentaje debe avanzar durante el prototipo.

Sin embargo:

IMPORTANTE:
Esta instalación NO realiza ninguna modificación real.

Debe utilizar un proceso simulado.

Preparar el código para que posteriormente el progreso pueda
ser sustituido por eventos reales de Calamares.

==========
10. PANTALLA 8 — FINALIZAR
==========

Mostrar una pantalla de instalación completada.

Debe incluir:

- confirmación de instalación;
- logo DylanOS;
- mensaje de éxito;
- resumen;
- recomendaciones;
- botón "Reiniciar ahora";
- opción de finalizar.

El botón "Reiniciar ahora" durante el prototipo debe mostrar
una confirmación o mensaje indicando que se trata de una simulación.

En la implementación real posteriormente deberá ejecutar

la acción correspondiente.

==========
11. DATOS DINÁMICOS
==========

Hay información que Figma Make NO puede obtener realmente.

NO inventar que esos datos son reales.

Crear una capa de datos simulados claramente separada de la interfaz.

Por ejemplo:

```
systemInfo = {
   device: "Inspiron 3421",
   cpu: "Intel Core i5-3210M",
   ram: "8 GB",
   gpu: "Intel HD Graphics 4000",
   storage: "512 GB"
}
```

Esto SOLO es información de demostración.

La arquitectura debe permitir reemplazar posteriormente:

systemInfo

por información proporcionada por Linux/Calamares.

==========
12. ARQUITECTURA DEL PROTOTIPO
==========

Separar:

UI
estado de instalación
datos simulados
validaciones
navegación

No colocar todos los datos directamente dentro de los componentes.

Crear un estado global del instalador.

Ejemplo conceptual:

```
installerState = {
    currentStep,
    language,
    keyboard,
    partitioning,
    user,
    systemInfo,
    installationProgress
}
```

La navegación debe utilizar este estado.

==========
13. COSAS QUE SÍ DEBE HACER EL PROTOTIPO
==========

✓ Navegación entre pantallas
✓ Botones funcionales
✓ Formularios
✓ Validaciones
✓ Selección de idioma
✓ Selección de teclado
✓ Selección de particionado simulado
✓ Creación de usuario simulada
✓ Resumen dinámico
✓ Progreso de instalación simulado
✓ Estados visuales
✓ Hover
✓ Focus
✓ Animaciones
✓ Diálogos de confirmación
✓ Responsive
✓ Persistencia del estado durante la sesión

==========
14. COSAS QUE NO DEBE SIMULAR COMO SI FUERAN REALES
==========

NO fingir que:

- está modificando el disco;
- está instalando paquetes reales;
- está cambiando el hostname real;
- está creando usuarios Linux;
- está cambiando contraseñas;
- está detectando hardware real;
- está reiniciando realmente el ordenador.

En esas partes utilizar datos simulados y dejar claramente preparada
la interfaz para una futura integración.

==========
15. PREPARACIÓN PARA LA IMPLEMENTACIÓN REAL
==========

El objetivo final es convertir este prototipo en el instalador real
de DylanOS utilizando Calamares.

Por eso:

- mantener los componentes organizados;
- mantener los datos separados de la UI;
- evitar código innecesariamente acoplado;
- utilizar componentes reutilizables;
- mantener nombres claros;
- documentar dónde existen datos simulados;
- marcar claramente los puntos que deberán conectarse con Calamares.

NO reemplazar Calamares.

Figma Make está creando el PROTOTIPO Y FRONTEND VISUAL.

Calamares será posteriormente responsable de las operaciones reales.

==========
16. RESULTADO FINAL
==========

Quiero un instalador completo que pueda recorrerse desde:

Bienvenido
→ Idioma
→ Teclado
→ Particionado
→ Usuario
→ Resumen
→ Instalando DylanOS
→ Finalizar

Debe sentirse como UNA SOLA APLICACIÓN coherente.

Analiza primero TODAS las imágenes adjuntas y respeta su composición,
jerarquía visual, colores, espaciado, tarjetas, sidebar, botones,
iconografía y estética.

No simplifiques el diseño innecesariamente.

La prioridad es:

1. Fidelidad visual
2. Experiencia de usuario
3. Funcionalidad del prototipo
4. Arquitectura preparada para datos reales
5. Facilidad de integración posterior con Calamares

==========

17. REQUISITO OBLIGATORIO DE FIDELIDAD VISUAL

==========

TODAS las pantallas del prototipo DEBEN mantener como mínimo un 90% de similitud visual con los bocetos proporcionados.

Los bocetos adjuntos son la REFERENCIA PRINCIPAL DEL DISEÑO.

NO deben tratarse simplemente como inspiración.

Antes de generar cada pantalla, analiza cuidadosamente su boceto correspondiente y reproduce:

- Composición
- Distribución de elementos
- Posición de los paneles
- Tamaños relativos
- Espaciado
- Márgenes
- Jerarquía visual
- Tipografía
- Tamaños de texto
- Colores
- Transparencias
- Bordes
- Radios de las esquinas
- Iconografía
- Botones
- Sidebar
- Tarjetas
- Ilustraciones
- Indicadores de progreso
- Elementos de navegación

El resultado debe ser reconociblemente el MISMO DISEÑO que aparece en los bocetos, pero convertido en una interfaz funcional.

NO simplifiques una pantalla si el boceto contiene más elementos.

NO cambies la distribución solamente porque otra distribución sea
más sencilla de programar.

NO sustituyas componentes visuales importantes por componentes
genéricos.

Si una funcionalidad dinámica no puede implementarse en Figma Make,
MANTÉN SU DISEÑO VISUAL exactamente como aparece en el boceto y
utiliza datos simulados.

La funcionalidad puede ser simulada cuando sea necesario,
pero la apariencia NO debe simplificarse.

OBJETIVO:

≥ 90% de fidelidad visual respecto a los bocetos.

La funcionalidad puede diferir temporalmente de la implementación
real de Calamares, pero el DISEÑO VISUAL debe mantenerse.

Antes de considerar terminado el prototipo, compara cada pantalla
generada con su boceto correspondiente y realiza los ajustes
necesarios para alcanzar al menos el 90% de similitud.

PRIORIDAD ABSOLUTA:

1. Fidelidad visual a los bocetos
2. Experiencia de usuario
3. Funcionalidad del prototipo
4. Preparación para integración con Calamares

==========

18. ESTRUCTURA Y ORGANIZACIÓN DEL CÓDIGO

==========

El proyecto DEBE estar organizado de forma clara, modular y fácil
de entender cuando descarguemos el código generado.

NO colocar toda la aplicación en un único archivo.

Utilizar una estructura similar a:

```
src/
├── components/
│   ├── Sidebar/
│   ├── TopBar/
│   ├── BottomNavigation/
│   ├── Buttons/
```

```
│   ├── Cards/
│   ├── Progress/
│   └── SystemInfo/
│
├── screens/
│   ├── Welcome/
│   ├── Language/
│   ├── Keyboard/
│   ├── Partitioning/
│   ├── UserCreation/
│   ├── Summary/
│   ├── Installation/
│   └── Finish/
│
├── data/
│   ├── mockSystemInfo
│   ├── languages
│   ├── keyboards
│   └── partitionExamples
│
├── state/
│   └── installerState
│
├── services/
│   └── simulatedInstallation
│
├── assets/
│   ├── logos/
│   ├── backgrounds/
│   ├── icons/
│   └── illustrations/
│
├── styles/
│   ├── colors
│   ├── typography
│   ├── spacing
│   └── effects
│
└── App
```

La estructura exacta puede adaptarse al framework utilizado por
Figma Make, pero DEBE conservar esta separación conceptual.

==========
18.1 NOMENCLATURA
==========

Utilizar nombres claros y descriptivos.

BUENO:

WelcomeScreen
LanguageScreen
KeyboardScreen
PartitioningScreen
UserCreationScreen
SummaryScreen
InstallationScreen
FinishScreen

EVITAR:

page1
page2
componentA
frame123
test
newComponent
temp

Los nombres deben permitir identificar inmediatamente qué hace cada archivo o componente.

==========
18.2 DATOS SIMULADOS
==========

Todos los datos que posteriormente tendrán que ser sustituidos por datos reales DEBEN estar claramente separados.

Por ejemplo:

mockSystemInfo
mockInstallationProgress
mockDiskData

NO mezclar datos simulados directamente dentro de los componentes visuales.

==========
18.3 MARCADO PARA FUTURA INTEGRACIÓN
==========

En el código, identificar claramente las partes que posteriormente deberán conectarse con Calamares/Linux.

Utilizar comentarios claros como:

// DYLANOS-INTEGRATION:
// Replace mock data with real Calamares system information.

o:

// DYLANOS-INTEGRATION:
// Connect this action to the real Calamares partitioning module.

o:

// DYLANOS-MOCK:
// This is simulated only for the Figma Make prototype.

Estos comentarios deben aparecer únicamente donde realmente sean necesarios.

==========
18.4 DOCUMENTACIÓN
==========

Crear un README.md que explique:

- Cómo está organizado el proyecto.
- Qué hace cada carpeta.
- Qué pantalla corresponde a cada componente.
- Cómo ejecutar el prototipo.
- Qué partes son simuladas.
- Qué partes deberán conectarse posteriormente a Calamares.
- Dónde están los recursos gráficos.
- Dónde modificar colores y tipografía.
- Dónde modificar los datos simulados.

También crear un archivo:

docs/CALAMARES_INTEGRATION.md

que enumere específicamente qué partes del prototipo tendrán que ser reemplazadas o conectadas cuando lo integremos con Calamares.

==========
18.5 OBJETIVO
==========

Cuando descarguemos el proyecto desde Figma Make, debemos poder abrirlo y entender rápidamente:

- dónde está cada pantalla;
- dónde está cada componente;
- dónde están los datos;
- dónde están los estilos;
- qué es simulado;
- qué debemos conectar a Calamares;
- qué archivos contienen los recursos visuales.

La organización del proyecto es TAN IMPORTANTE como su apariencia.

Priorizar código limpio, modular, mantenible y fácil de adaptar.